

Scenario 6 — Basic Plan, Claim Exceeds Coverage, Blocked

Full mechanics: see `Step-3`. This is the coverage-cap check, not the premium check (contrast with Scenario 5 — different guard, different error).

Values used

	Value
Policyholder	Alice, <code>0x1111...aaaa</code> (a second, separate Basic policy — <code>policyId = 4</code> , continuing the ID sequence from earlier scenarios in this document set)
templateId	1 (Basic)
coverageAmount	<code>2_000 ether</code>
Premium paid	11 ether — current, not lapsed
Claim amount attempted	<code>2_500 ether</code> (\$2,500 — exceeds the \$2,000 cap)
Result	Reverts with <code>ClaimExceedsPolicyCoverage</code>

Walkthrough (deltas)

- **Step 2:** identical mechanics to Scenario 1, just a fresh `policyId`.
- **Step 3, `submitClaim`:**
 1. `2_500 ether <= maxPayoutPerClaim` — this is a global, contract- wide cap (configured much higher, e.g. `1_000_000 ether`), unrelated to any specific policy's coverage — passes.
 2. `getPolicy / holder / date-range` checks — all pass, identical pattern to every other scenario.
 3. `isPremiumCurrent` — `true`, premium is paid up.
 4. `_consumedCoverage(4)` — this is Alice's first claim on this particular policy, so it returns `0`.
 5. The actual check: `0 + 2_500 ether <= 2_000 ether` — false.
 6. `if (!(consumedSoFar + amount <= coverageAmount)) revert ClaimExceedsPolicyCoverage();` — reverts here.

End state

```
claims[?]: does not exist - reverted before any Claim struct was written
```

The more subtle version of this same check — cumulative, not just single-claim

This same guard also catches a claim that's individually within coverage but pushes the cumulative total over the cap, counting every non-`Rejected` claim on the policy (`Submitted`, `UnderReview`, `Approved`, and `Paid` all still count — only `Rejected` claims are excluded from the running total). Concretely: if Alice had an earlier `Approved-but-not-yet-Paid` claim for `1_500 ether` still open on this same \$2,000-coverage policy, a second claim for even `600 ether` would revert — `1_500 + 600 = 2_100 > 2_000` — despite `600 ether` alone being well under the cap. This is `_consumedCoverage`'s entire purpose (see `Step-3`'s full explanation): a policy's coverage is a shared budget across every claim against it, not a per-claim allowance.

